should be the heart of the system, with lesser concerns being plugged in to them. The business rules should be the most independent and reusable code in the system.

1. This is Ivar Jacobson's name for this concept (I. Jacobson et al., *Object Oriented Software Engineering*, Addison-Wesley, 1992).
2. Ibid.

21
SCREAMING ARCHITECTURE

Imagine that you are looking at the blueprints of a building. This document, prepared by an architect, provides the plans for the building. What do these plans tell you?

If the plans you are viewing are for a single-family residence, then you'll likely see a front entrance, a foyer leading to a living room, and perhaps a dining room. There will likely be a kitchen a short distance away, close to the dining room. Perhaps there is a dinette area next to the kitchen, and probably a family room close to that. When you looked at those plans, there would be no question that you were looking at a single family home. The architecture would scream: "HOME."

Now suppose you were looking at the architecture of a library. You would likely see a grand entrance, an area for check-in/out clerks, reading areas, small conference rooms, and gallery after gallery capable of holding bookshelves for all the books in

the library. That architecture would scream: “LIBRARY.”

So what does the architecture of your application scream? When you look at the top-level directory structure, and the source files in the highest-level package, do they scream “Health Care System,” or “Accounting System,” or “Inventory Management System”? Or do they scream “Rails,” or “Spring/Hibernate,” or “ASP”?

THE THEME OF AN ARCHITECTURE

Go back and read Ivar Jacobson’s seminal work on software architecture: *Object Oriented Software Engineering*. Notice the subtitle of the book: *A Use Case Driven Approach*. In this book Jacobson makes the point that software architectures are structures that support the use cases of the system. Just as the plans for a house or a library scream about the use cases of those buildings, so should the architecture of a software application scream about the use cases of the application.

Architectures are not (or should not be) about frameworks. Architectures should not be supplied by frameworks. Frameworks are tools to be used, not architectures to be conformed to. If your architecture is based on frameworks, then it cannot be based on your use cases.

THE PURPOSE OF AN ARCHITECTURE

Good architectures are centered on use cases so that architects can safely describe the structures that support those use cases without committing to frameworks, tools, and environments. Again, consider the plans for a house. The first concern of the architect is to make sure that the house is usable—not to ensure that the house is made of bricks. Indeed, the architect takes pains to ensure that the homeowner can make decisions about the exterior material (bricks, stone, or cedar) later, after the plans ensure that the use cases are met.

A good software architecture allows decisions about frameworks, databases, web servers, and other environmental issues and tools to be deferred and delayed. *Frameworks are options to be left open.* A good architecture makes it unnecessary to decide on Rails, or Spring, or Hibernate, or Tomcat, or MySQL, until much later in the project. A good architecture makes it easy to change your mind about those decisions, too. A good architecture emphasizes the use cases and decouples them from peripheral concerns.

BUT WHAT ABOUT THE WEB?

Is the web an architecture? Does the fact that your system is delivered on the web dictate the architecture of your system? Of course not! The web is a delivery mechanism—an IO device—and your application architecture should treat it as such. The fact that your application is delivered over the web is a detail and should not dominate your system structure. Indeed, the decision that your application will be delivered over the web is one that you should defer. Your system architecture should be as ignorant as possible about how it will be delivered. You should be able to deliver it as a console app, or a web app, or a thick client app, or even a web service app, without undue complication or change to the fundamental architecture.

FRAMEWORKS ARE TOOLS, NOT WAYS OF LIFE

Frameworks can be very powerful and very useful. Framework authors often believe very deeply in their frameworks. The examples they write for how to use their frameworks are told from the point of view of a true believer. Other authors who write about the framework also tend to be disciples of the true belief. They show you the way to use the framework. Often they assume an all-encompassing, all-pervading, let-the-framework-do-everything position.

> *This is not the position you want to take.*

Look at each framework with a jaded eye. View it skeptically. Yes, it might help, but at what cost? Ask yourself how you should use it, and how you should protect yourself from it. Think about how you can preserve the use-case emphasis of your architecture. Develop a strategy that prevents the framework from taking over that architecture.

TESTABLE ARCHITECTURES

If your system architecture is all about the use cases, and if you have kept your frameworks at arm’s length, then you should be able to unit-test all those use cases without any of the frameworks in place. You shouldn’t need the web server running to run your tests. You shouldn’t need the database connected to run your tests. Your Entity objects should be plain old objects that have no dependencies on frameworks

or databases or other complications. Your use case objects should coordinate your Entity objects. Finally, all of them together should be testable in situ, without any of the complications of frameworks.

CONCLUSION

Your architecture should tell readers about the system, not about the frameworks you used in your system. If you are building a health care system, then when new programmers look at the source repository, their first impression should be, “Oh, this is a heath care system.” Those new programmers should be able to learn all the use cases of the system, yet still not know how the system is delivered. They may come to you and say:

“We see some things that look like models—but where are the views and controllers?”

And you should respond:

“Oh, those are details that needn’t concern us at the moment. We’ll decide about them later.”

22
THE CLEAN ARCHITECTURE

Over the last several decades we've seen a whole range of ideas regarding the architecture of systems. These include:

- Hexagonal Architecture (also known as Ports and Adapters), developed by Alistair Cockburn, and adopted by Steve Freeman and Nat Pryce in their wonderful book *Growing Object Oriented Software with Tests*
- DCI from James Coplien and Trygve Reenskaug
- BCE, introduced by Ivar Jacobson from his book *Object Oriented Software Engineering: A Use-Case Driven Approach*

Although these architectures all vary somewhat in their details, they are very similar. They all have the same objective, which is the separation of concerns. They all

achieve this separation by dividing the software into layers. Each has at least one layer for business rules, and another layer for user and system interfaces.

Each of these architectures produces systems that have the following characteristics:

- *Independent of frameworks.* The architecture does not depend on the existence of some library of feature-laden software. This allows you to use such frameworks as tools, rather than forcing you to cram your system into their limited constraints.
- *Testable.* The business rules can be tested without the UI, database, web server, or any other external element.
- *Independent of the UI.* The UI can change easily, without changing the rest of the system. A web UI could be replaced with a console UI, for example, without changing the business rules.
- *Independent of the database.* You can swap out Oracle or SQL Server for Mongo, BigTable, CouchDB, or something else. Your business rules are not bound to the database.
- *Independent of any external agency.* In fact, your business rules don't know anything at all about the interfaces to the outside world.

The diagram in Figure 22.1 is an attempt at integrating all these architectures into a single actionable idea.

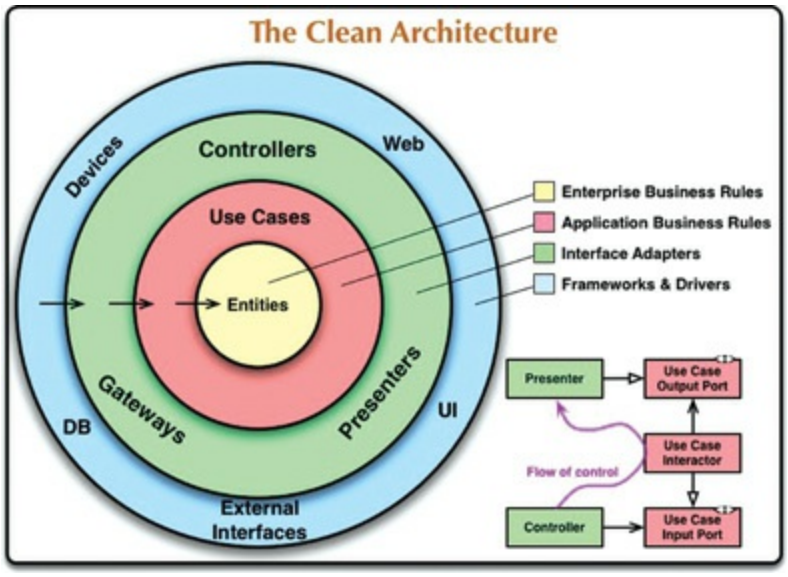

Figure 22.1 The clean architecture

THE DEPENDENCY RULE

The concentric circles in Figure 22.1 represent different areas of software. In general, the further in you go, the higher level the software becomes. The outer circles are mechanisms. The inner circles are policies.

The overriding rule that makes this architecture work is the *Dependency Rule*:

> *Source code dependencies must point only inward, toward higher-level policies.*

Nothing in an inner circle can know anything at all about something in an outer circle. In particular, the name of something declared in an outer circle must not be mentioned by the code in an inner circle. That includes functions, classes, variables, or any other named software entity.

By the same token, data formats declared in an outer circle should not be used by an inner circle, especially if those formats are generated by a framework in an outer circle. We don't want anything in an outer circle to impact the inner circles.

ENTITIES

Entities encapsulate enterprise-wide Critical Business Rules. An entity can be an object with methods, or it can be a set of data structures and functions. It doesn't matter so long as the entities can be used by many different applications in the enterprise.

If you don't have an enterprise and are writing just a single application, then these entities are the business objects of the application. They encapsulate the most general and high-level rules. They are the least likely to change when something external changes. For example, you would not expect these objects to be affected by a change to page navigation or security. No operational change to any particular application should affect the entity layer.

USE CASES

The software in the use cases layer contains *application-specific* business rules. It encapsulates and implements all of the use cases of the system. These use cases orchestrate the flow of data to and from the entities, and direct those entities to use their Critical Business Rules to achieve the goals of the use case.

We do not expect changes in this layer to affect the entities. We also do not expect this layer to be affected by changes to externalities such as the database, the UI, or

any of the common frameworks. The use cases layer is isolated from such concerns.

We do, however, expect that changes to the operation of the application will affect the use cases and, therefore, the software in this layer. If the details of a use case change, then some code in this layer will certainly be affected.

INTERFACE ADAPTERS

The software in the interface adapters layer is a set of adapters that convert data from the format most convenient for the use cases and entities, to the format most convenient for some external agency such as the database or the web. It is this layer, for example, that will wholly contain the MVC architecture of a GUI. The presenters, views, and controllers all belong in the interface adapters layer. The models are likely just data structures that are passed from the controllers to the use cases, and then back from the use cases to the presenters and views.

Similarly, data is converted, in this layer, from the form most convenient for entities and use cases, to the form most convenient for whatever persistence framework is being used (i.e., the database). No code inward of this circle should know anything at all about the database. If the database is a SQL database, then all SQL should be restricted to this layer—and in particular to the parts of this layer that have to do with the database.

Also in this layer is any other adapter necessary to convert data from some external form, such as an external service, to the internal form used by the use cases and entities.

FRAMEWORKS AND DRIVERS

The outermost layer of the model in Figure 22.1 is generally composed of frameworks and tools such as the database and the web framework. Generally you don’t write much code in this layer, other than glue code that communicates to the next circle inward.

The frameworks and drivers layer is where all the details go. The web is a detail. The database is a detail. We keep these things on the outside where they can do little harm.

ONLY FOUR CIRCLES?

The circles in Figure 22.1 are intended to be schematic: You may find that you need more than just these four. There's no rule that says you must always have just these four. However, the Dependency Rule always applies. Source code dependencies always point inward. As you move inward, the level of abstraction and policy increases. The outermost circle consists of low-level concrete details. As you move inward, the software grows more abstract and encapsulates higher-level policies. The innermost circle is the most general and highest level.

CROSSING BOUNDARIES

At the lower right of the diagram in Figure 22.1 is an example of how we cross the circle boundaries. It shows the controllers and presenters communicating with the use cases in the next layer. Note the flow of control: It begins in the controller, moves through the use case, and then winds up executing in the presenter. Note also the source code dependencies: Each points inward toward the use cases.

We usually resolve this apparent contradiction by using the Dependency Inversion Principle. In a language like Java, for example, we would arrange interfaces and inheritance relationships such that the source code dependencies oppose the flow of control at just the right points across the boundary.

For example, suppose the use case needs to call the presenter. This call must not be direct because that would violate the Dependency Rule: No name in an outer circle can be mentioned by an inner circle. So we have the use case call an interface (shown in Figure 22.1 as "use case output port") in the inner circle, and have the presenter in the outer circle implement it.

The same technique is used to cross all the boundaries in the architectures. We take advantage of dynamic polymorphism to create source code dependencies that oppose the flow of control so that we can conform to the Dependency Rule, no matter which direction the flow of control travels.

WHICH DATA CROSSES THE BOUNDARIES

Typically the data that crosses the boundaries consists of simple data structures. You can use basic structs or simple data transfer objects if you like. Or the data can simply be arguments in function calls. Or you can pack it into a hashmap, or construct it into an object. The important thing is that isolated, simple data structures are passed across the boundaries. We don't want to cheat and pass Entity objects or